

Prabhupada World Bhakti Vriksha Management App

Final Product Requirements Document (PRD) for AntigraVity + Cloudflare + Firebase
Draft v1.0 for Review

Purpose of this document

This document consolidates all discussed requirements, safeguards, workflows, modules, data collections, automations, acceptance criteria, and AntigraVity development instructions for the Prabhupada World Bhakti Vriksha Management App.

Item	Details
Prepared for	Prabhupada World
Primary use	Review and master prompt/specification for AntigraVity development
Recommended stack	AntigraVity for development, Cloudflare Pages/Workers for hosting and edge functions, Firebase Auth/Firestore/Cloud Functions for app data and core automations
Document date	2026-06-16

Contents

- 1. Executive Summary
- 2. Product Vision and Principles
- 3. Scope and Non-Goals
- 4. Recommended Technology Architecture
- 5. User Roles and Access Model
- 6. Privacy, Consent and Data Governance
- 7. Installable Web App and Signup Flow
- 8. New Member Approval Workflow
- 9. Member Master Profile
- 10. Bhakti Vriksha Groups and Zones
- 11. Daily Attendance Tracking
- 12. Daily Sadhana Tracking
- 13. Seva Management
- 14. One-to-One Call Tracker
- 15. Follow-up Management
- 16. Meetings and Minutes of Meeting
- 17. Resources Module
- 18. Quiz Management
- 19. Dashboards and Reports
- 20. Notifications and WhatsApp Integration
- 21. Firestore Collections and Data Model
- 22. Automations
- 23. Security, Audit Logs, Backup and Restore
- 24. Operational Safeguards and Loophole Fixes
- 25. MVP Scope and Future Roadmap
- 26. Development Instructions for AntigraVity
- 27. Acceptance Criteria
- 28. Appendices: Status Values and Master Prompt Summary

1. Executive Summary

Prabhupada World needs a centralized, secure, role-based Bhakti Vriksha management system to replace scattered Google Sheets, manual WhatsApp coordination, and fragmented follow-up tracking. The app should become a daily care and engagement system for group leaders, coordinators, admins, mentors, and members.

- Members should self-sign up through an installable web app and wait for approval.
- Admins should review new signups, assign members to a group/leader, and track approval stages.
- Group leaders should accept assigned members before members receive app access and WhatsApp group joining information.
- Daily attendance, daily sadhana, seva assignments, one-to-one calls, follow-ups, meeting minutes, resources, and quizzes should be handled in one place.
- Permissions, privacy, audit logs, duplicate management, backups, and dashboard summary records should be built from the beginning.

Core success question

Will this feature help leaders care for devotees better? If yes, it belongs in the system. If not, keep it for later.

2. Product Vision and Principles

The app should not be a generic CRM. It should be a Bhakti Vriksha operating system built around spiritual care, accountability, daily engagement, and safe data handling.

2.1. Design Principles

- Simple enough for non-technical devotees to use.
- Mobile-first and installable like an app through Progressive Web App support.
- Secure by default: no unauthenticated access to internal data.
- Role-based by design: members, leaders, coordinators, and admins see only relevant information.
- Database-first: avoid recreating spreadsheet chaos inside a prettier interface.
- Daily-care oriented: attendance, sadhana, seva, and follow-ups should help leaders act quickly.
- Respectful and devotional, but professional and uncluttered.

2.2. Suggested UI Tone

- Light background with soft saffron/gold accents and deep blue or maroon secondary accents.
- Large buttons, clear forms, clean dashboards, status badges, and minimal visual clutter.
- Cards for dashboard summaries; tables for admin data; search and filters wherever volume is expected.

3. Scope and Non-Goals

3.1. In Scope for MVP

- Progressive Web App setup and responsive UI.
- Firebase Auth login and email verification.
- Self-signup with profile and sadhana details.
- Waiting for Approval screen.
- Admin assignment and leader acceptance workflow.
- Member master profile, group/zone management, daily attendance, daily sadhana, seva, call logs, follow-ups, meetings, resources, quizzes, notifications, audit logs, reports, and dashboards.
- Role-based route protection and Firestore Security Rules.
- CSV import/export for admins.
- Manual WhatsApp fallback even if API integration is delayed.

3.2. Out of Scope for MVP

- Native Android/iOS app.

- Payment integration.
- Full course/LMS system.
- Certificate generation.
- Advanced AI summaries and recommendations.
- Full multi-language UI.
- Complete WhatsApp API dependency as a blocker for launch.

4. Recommended Technology Architecture

Layer	Tool	Purpose
Development	Antigravity	Generate and maintain the application code through agentic coding workflows.
Frontend	React or Next.js with TypeScript	Main user interface for members, leaders, coordinators, and admins.
Hosting	Cloudflare Pages	Host the installable web app with fast global delivery.
Edge/API	Cloudflare Workers	Webhook receivers, lightweight APIs, selected scheduled jobs, and external integrations.
Authentication	Firebase Authentication	Email/password login, email verification, and later phone login if needed.
Database	Cloud Firestore	Main application database.
Backend automations	Firebase Cloud Functions	Core secure automations, Firestore triggers, role/claim updates, and privileged admin tasks.
Security	Firebase Security Rules + App Check	Restrict reads/writes and protect backend access.
Notifications	In-app, email, WhatsApp later	Send reminders, approval messages, resource alerts, quiz notifications, and follow-up alerts.

4.1. Architecture Recommendation

Antigravity

- > React / Next.js / TypeScript app
- > Cloudflare Pages
- > Firebase Auth + Firestore
- > Firebase Cloud Functions for core automations
- > Cloudflare Workers for webhooks / edge APIs / selected cron jobs
- > WhatsApp Cloud API or provider for WhatsApp messages

Important implementation note

Use Firebase Cloud Functions for privileged Firebase Admin SDK tasks such as assigning roles, updating custom claims, and trusted backend writes. Cloudflare Workers are useful, but should not become the place where Firebase admin secrets are exposed casually.

5. User Roles and Access Model

Every user should have a role and should be routed to a dashboard appropriate for that role. Frontend hiding is not enough: Firestore Security Rules and backend checks must enforce access.

Role	Primary Access
Super Admin	Full system access, role changes, settings, all data, all reports, export, security configuration.
Admin	Operational management: members, groups, approvals, quizzes, resources, reports; limited critical deletion.
Coordinator	Zone/city-level view and management for assigned area only.
Bhakti Vriksha Leader	Assigned group members, attendance, sadhana, seva assignment/review, calls, follow-ups, group dashboard.
Assistant Leader	Limited leader support access: attendance, sadhana updates, basic remarks, optional seva review.

Mentor/Caller	Assigned members only; call logs, follow-up notes, and support remarks.
Member	Own profile, daily sadhana, own attendance, assigned seva, visible resources, active quizzes, group details.
Data Entry Volunteer	Limited member creation/import/update access; no sensitive remarks or role changes.

5.1. Role-Based Access Matrix Summary

Feature Area	Admin/Super Admin	Coordinator	Leader/Assistant	Member
Members	All members	Assigned zone	Assigned group/assigned members	Own profile only
Attendance	All data	Assigned zone	Mark/view assigned group	Own history only
Sadhana	All data	Assigned zone	View/update assigned group	Submit/view own only
Seva	All assignments	Assigned zone	Assign/review group seva	Update own assigned seva
Meetings	Create/approve/view by visibility	Create/view if allowed	View/add if allowed	Only if explicitly visible
Resources	Upload/publish for all	Upload for zone if allowed	Upload for group if allowed	View visible resources only
Quizzes	Create/publish/view all results	Zone if allowed	Group quizzes if allowed	Attempt visible quizzes only
Exports	Allowed	Limited if allowed	Usually no	No

6. Privacy, Consent and Data Governance

The app will store personal and sensitive community information: phone numbers, WhatsApp numbers, family remarks, spiritual practice history, attendance, seva, counselling remarks, and mentor notes. Privacy controls are therefore a core requirement.

6.1. Consent Requirements

- Signup must include a clear consent checkbox before submission.
- Consent should cover spiritual guidance, group allocation, communication, follow-up, and Bhakti Vriksha participation.
- Member should be able to view submitted profile summary while waiting for approval.
- The app should include a Privacy Policy page and basic data handling explanation.
- Members should be able to request data correction, export, or deletion/deactivation.

Recommended consent fields:

```

consentGiven
consentGivenAt
privacyPolicyAcceptedAt
communicationConsentWhatsApp
communicationConsentEmail
communicationConsentPhone
dataDeletionRequested
dataExportRequested

```

6.2. Sensitive Remarks Visibility

- Private counselling or family-related remarks should never be visible to ordinary members.
- Default remark visibility should not be Member Visible.
- Visibility options: Admin Only, Coordinator and Above, Assigned Leader Only, Assigned Mentor Only, Member Visible.
- Sensitive remark creation and edits must be audit logged.

7. Installable Web App and Signup Flow

7.1. Progressive Web App Requirements

- The app should be installable on mobile and desktop.
- Include app icon, splash screen, responsive design, secure HTTPS, and fast loading.
- Basic shell may be offline-friendly, but sensitive data should depend on authenticated access.
- Users may receive either the app link or install link.

7.2. New User Signup Fields

Category	Fields
Personal Details	Full name, preferred name, gender, age/date of birth, phone, WhatsApp number, email, city, state, country, address, language, occupation, company/college, marital status, family details/remarks, source.
Spiritual/Sadhana Details	Current chanting, daily rounds, book reading, current book, class attendance, spiritual routine, ISKCON/HKM connection, mentor, interest in Bhakti Vriksha, interest in seva, preferred session time, questions/remarks.
Consent	Consent checkbox and communication consent flags.

7.3. Email Verification

- Use Firebase Auth email/password signup.
- Send email verification immediately after signup.
- Until email is verified, show: "Please verify your email to continue."
- Phone/WhatsApp verification can be Phase 2; for MVP, admin may verify manually.

8. New Member Approval Workflow

8.1. Approval Stages

Signed Up - Email Not Verified
 Email Verified - Profile Not Submitted
 Profile Submitted - Waiting for Admin Approval
 Assigned to Leader - Waiting for Leader Approval
 Approved - Active
 Rejected
 On Hold
 Inactive
 Archived

8.2. Full Approval Flow

User receives app/install link
 -> User installs/opens web app
 -> User signs up
 -> User verifies email
 -> User fills personal and sadhana details
 -> User submits profile
 -> User sees Waiting for Approval
 -> Admin receives new signup alert
 -> Admin reviews profile and duplicate warnings
 -> Admin assigns member to group leader
 -> Group leader receives approval request
 -> Group leader accepts member
 -> System marks member Approved and Active
 -> System sends WhatsApp group link or creates manual send task
 -> Member gets active dashboard access

8.3. Waiting for Approval Screen

Hare Krishna!

Thank you for registering for Prabhupada World Bhakti Vriksha.
Your profile has been submitted successfully and is currently waiting for approval.
Our team will review your details and assign you to a suitable Bhakti Vriksha group.
You will receive further instructions once your registration is approved.

8.4. Admin Actions

- View new signups waiting for approval.
- View submitted profile and sadhana details.
- Check duplicate phone/email/WhatsApp/name-city matches.
- Edit or correct submitted details if required.
- Assign zone, group, and group leader.
- Reject or place on hold with reason.
- Add admin remark and notify assigned leader.

8.5. Leader Actions

- View assigned member approval requests.
- View profile, sadhana details, and admin remarks.
- Accept the member, request admin review, reject with reason, or put on hold.
- On acceptance, system marks the member Approved and Active if admin assignment is complete.

8.6. Required Member Approval Fields

approvalStatus
emailVerified
phoneVerified
submittedAt
adminReviewedBy
adminReviewedAt
assignedGroupId
assignedLeaderId
assignedAt
leaderApprovalStatus
leaderApprovedBy
leaderApprovedAt
approvalCompletedAt
rejectionReason
onHoldReason
whatsappInviteSent
whatsappInviteSentAt

9. Member Master Profile

Each devotee/member should have one central profile. This replaces scattered Google Sheets and becomes the single source of truth.

9.1. Member Status Values

New
Active
Needs Follow-up
Temporarily Paused
Inactive
No Response
Not Interested
Shifted to Another Group
Duplicate

Archived
Reactivated

9.2. Member Profile Fields

Field Group	Fields
Basic	memberId, fullName, preferredName, gender, age, ageGroup, phone, alternatePhone, WhatsApp number, email, city, state, country, address, languagePreference, occupation, companyOrCollege, maritalStatus, familyRemarks.
PW/BV	source, dateOfRegistration, dateOfJoining, membershipType, currentStage, assignedGroupId, assignedLeaderId, assignedMentorId, assignedCallerId, status, seriousnessLevel, interestLevel.
Spiritual	chantingRounds, readingHabit, hearingHabit, attendsMorningClass, attendsEveningClass, connectedToISKCONOrHKM, spiritualQuestions, sevaInterest, bookReadingStatus.
Tracking	lastContactedAt, nextFollowUpAt, lastAttendanceAt, lastSadhanaSubmittedAt, lastQuizAttemptAt, lastSevaUpdateAt.
System	createdAt, createdBy, updatedAt, updatedBy, isDeleted.

9.3. Member Timeline

- Every important member event should be stored in a timeline: signup, email verification, approval, group join, first attendance, first sadhana, first seva, calls, inactive/reactivated, transfers.
- Collection: memberTimeline.
- Fields: timelineId, memberId, eventType, eventTitle, eventDescription, createdBy, createdAt, relatedEntityType, relatedEntityId.

10. Bhakti Vriksha Groups and Zones

10.1. Group Fields

groupId
 groupName
 zoneId
 city
 area
 leaderId
 assistantLeaderIds
 mentorIds
 meetingDay
 meetingStartTime
 meetingEndTime
 meetingMode
 meetingLocation
 meetingLink
 language
 status
 whatsappGroupName
 whatsappGroupLink
 whatsappGroupDescription
 whatsappLinkActive
 createdAt
 updatedAt

10.2. Group Transfer Workflow

- Do not simply overwrite assignedGroupId when a member shifts groups.
- Maintain groupTransferHistory with old group, new group, requested by, approved by, reason, transfer date, old leader remark, and new leader remark.

- Transfer should update current group only after approval and should preserve historical records.

10.3. Temporary/Substitute Leader Access

- Allow temporary access when a leader is travelling, sick, shifted, or overloaded.
- Temporary access must have accessStartDate, accessEndDate, reason, grantedBy, and status.
- This prevents password sharing.

11. Daily Attendance Tracking

Attendance must be daily, not weekly. Sessions may represent daily BV sessions, morning classes, evening reading sessions, Jigyasa, RDUa, mentor sessions, seva meetings, orientations, and special events.

11.1. Collections

```
sessions
dailyAttendance
```

11.2. Session Fields

```
sessionId
title
sessionType
groupId
zoneId
date
startTime
endTime
speaker
mode
location
meetingLink
createdBy
createdAt
updatedAt
status
```

11.3. Daily Attendance Fields

```
attendanceId
sessionId
memberId
groupId
zoneId
date
status
markedBy
markedAt
remarks
createdAt
updatedAt
```

11.4. Attendance Status Values

```
Present
Absent
Late
Excused
Unknown
```

11.5. Attendance Rules and Grace Periods

- Leader or authorized assistant marks daily attendance for assigned group.
- Member can view own attendance history only.

- Duplicate attendance for same session/member should be prevented using ID pattern sessionId_memberId.
- Leader can edit attendance for last X days; older corrections require admin permission.
- Late/backdated attendance edits must be logged.
- Grace period should be configurable.

12. Daily Sadhana Tracking

Sadhana tracking must be daily. Members should submit their own daily sadhana, while leaders/admins can help update when required.

12.1. Collection and Fields

```
dailySadhanaLogs:
  sadhanaLogId
  memberId
  groupId
  zoneId
  date
  submittedBy
  submittedAt
  chantingRounds
  readingDone
  readingMinutes
  hearingDone
  hearingMinutes
  morningClassAttended
  eveningClassAttended
  bookReadingDone
  bookName
  sevaDone
  associationDone
  regulativePrinciplesFollowed
  challengesFaced
  needsHelp
  remarks
  visibility
  submittedOnTime
  submittedLate
  editedAfterSubmission
  createdAt
  updatedAt
```

12.2. Daily Sadhana Form

- Date.
- How many rounds did you chant today?
- Did you read today? If yes, how many minutes?
- Did you hear any class/lecture today? If yes, how many minutes?
- Did you attend morning class?
- Did you attend evening class?
- Did you read any book? Book name if applicable.
- Did you do any seva today?
- Did you take devotee association today?
- Were you able to follow your spiritual commitments today?
- Are you facing any difficulty?
- Do you need help from a mentor/leader?
- Remarks.

12.3. Submission Rules

- One sadhana log per member per date. Recommended ID: memberId_YYYY-MM-DD.

- Member can edit today's sadhana until a configurable cutoff time.
- Older corrections require leader/admin permissions and must be logged.
- If needsHelp is true, create follow-up automatically.
- Dashboards should use summary documents where possible to control Firebase reads.

13. Seva Management

Seva should be a full module, not a single field. Leaders should assign seva, members should update progress, and admins/leaders should track time spent, status, blockers, and reviews.

13.1. Seva Categories

Calling
 Documentation
 Coding
 Graphic Design
 Video Editing
 Content Writing
 Social Media
 Data Entry
 Book Distribution
 Event Support
 WhatsApp Follow-up
 Mentoring
 Teaching Support
 Quiz Creation
 Course Support
 Translation
 Proofreading
 Technical Support
 App Testing
 Community Management
 Devotee Care
 Other

13.2. Seva Assignment Fields

sevaAssignmentId
 title
 description
 sevaType
 assignedBy
 assignedToMemberId
 assignedToUserId
 groupId
 zoneId
 priority
 status
 expectedStartDate
 expectedEndDate
 estimatedHours
 instructions
 referenceLink
 createdAt
 updatedAt
 completedAt
 isRecurring
 recurrencePattern
 visibility

13.3. Seva Status Values

Assigned
 Not Started
 Started
 In Progress
 Submitted
 Needs Review
 Changes Requested
 Completed
 Blocked
 Paused
 Cancelled

13.4. Seva Log Fields

sevaLogId
 sevaAssignmentId
 memberId
 userId
 groupId
 zoneId
 date
 statusUpdate
 timeSpentMinutes
 workDoneDescription
 completionPercentage
 blocker
 needsHelp
 submittedBy
 submittedAt
 reviewedBy
 reviewedAt
 reviewRemark
 createdAt
 updatedAt

13.5. Seva Workflow

Leader assigns seva -> Member sees assigned seva -> Member submits progress update -> Leader reviews -> Leader approves / requests changes / marks complete -> Dashboard and contribution summary update.

13.6. Seva Dashboard Metrics

- Active assignments, completed sevas, pending reviews, blocked sevas, high-priority sevas, overdue sevas.
- Total seva hours logged, category-wise contribution, group-wise contribution, zone-wise contribution, top active volunteers.
- Members who have not updated seva and sevas due today.

14. One-to-One Call Tracker

The call tracker stores personal calls, counselling, follow-ups, and remarks in a structured way.

14.1. Call Log Fields

callId
 memberId
 callerId
 groupId
 callDate
 callType
 callStatus
 discussionSummary

memberConcern
 spiritualConcern
 familyConcern
 sadhanaConcern
 nextStep
 nextFollowUpDate
 priority
 privateRemark
 visibleToRoles
 createdAt
 updatedAt

14.2. Call Status Values

Connected
 Did Not Answer
 Busy
 Switched Off
 Wrong Number
 Call Back Later
 Not Interested
 Follow-up Required
 Resolved

- Private call remarks must never be visible to normal members.

15. Follow-up Management

15.1. Follow-up Fields

followUpId
 memberId
 assignedTo
 groupId
 reason
 dueDate
 status
 priority
 createdFrom
 createdAt
 completedAt
 completionRemark

15.2. Follow-up Reasons

Missed Session
 Sadhana Not Submitted
 Low Attendance
 Requested Call
 New Member Onboarding
 Quiz Not Completed
 Spiritual Doubt
 Membership Discussion
 Reconnect
 Meeting Action Item
 Seva Blocked
 Other

- Every follow-up must have an owner. No owner means no accountability.
- Dashboards should show due today, overdue, due this week, high-priority, by leader, and by group.
- Meeting action items and blocked seva may create follow-ups automatically.

16. Meetings and Minutes of Meeting

The app must store minutes of weekly facilitator meetings and executive committee meetings, with visibility control and action item tracking.

16.1. Collections

meetings
meetingMinutes
meetingActionItems

16.2. Meeting Types

Weekly Facilitator Meeting
Executive Committee Meeting
Group Leader Meeting
Seva Team Meeting
Training Meeting
Review Meeting
Other

16.3. Meeting Fields

meetingId
title
meetingType
date
startTime
endTime
mode
location
meetingLink
createdBy
attendees
visibleToRoles
visibleToGroupIds
visibleToZoneIds
status
createdAt
updatedAt

16.4. Minutes Fields

minutesId
meetingId
summary
discussionPoints
decisionsTaken
concernsRaised
nextSteps
preparedBy
approvedBy
approvedAt
visibility
status
createdAt
updatedAt

16.5. Minutes Approval Flow

Draft
Submitted for Approval
Approved

Published
Archived

16.6. Meeting Action Items

- Action items should have title, description, assigned owner, due date, priority, status, completion remark, and completion date.
- Action item statuses: Pending, In Progress, Completed, Overdue, Cancelled.
- Allow creating a Seva Assignment or Follow-up from a meeting action item.

17. Resources Module

Admins/team leaders should upload resources and select exactly who can view them.

17.1. Resource Fields

```
resourceId
title
description
resourceType
categoryId
fileUrl
externalLink
uploadedBy
visibleToRoles
visibleToGroupIds
visibleToZoneIds
visibleToUserIds
visibleToMemberIds
isPublicToApprovedMembers
status
versionNumber
replacesResourceId
isLatestVersion
changeSummary
createdAt
updatedAt
expiresAt
```

17.2. Resource Types and Categories

Resource Types	Initial Categories
PDF, Document, Image, Video Link, Audio Link, Google Drive Link, Template, Guideline, Class Notes, Meeting Notes, Other	Leader Training, Facilitator Resources, Sadhana Resources, Jigyasa Resources, Bhagavad Gita Resources, BV Guidelines, Seva Training, Calling Scripts, Documentation Templates, Graphic Templates, Coding/Technical Guidelines, Quiz Resources, Meeting Notes, General

17.3. Resource Visibility and Acknowledgement

- Visibility options: all approved members, all leaders, all coordinators, specific zone, specific group, specific users, specific roles, admin only.
- Draft resources should not be visible to members.
- Archived resources should not appear in active lists.
- Important resources should have an acknowledgement option: "I have read this."
- Resource versioning should track latest version and change summary.

18. Quiz Management

The quiz tab should be visible, but quizzes should appear only when leader/admin publishes or makes them visible. Quizzes should support time limits and countdown timers.

18.1. Quiz Visibility Rules

- Quiz tab always visible to approved members.
- Only active/visible quizzes appear inside the tab and on dashboard.
- Admin can create/publish quizzes for all or selected groups/zones/members.
- Leader can create/publish quizzes for assigned group if permission is given.
- Quiz can be unpublished, closed, or archived.
- Publishing should immediately show the quiz on assigned members' dashboards.

18.2. Quiz Fields

```

quizId
title
description
relatedCourse
createdBy
status
passingScore
isVisible
publishedBy
publishedAt
visibleFrom
visibleUntil
visibleToRoles
visibleToGroupIds
visibleToZoneIds
visibleToMemberIds
hasTimeLimit
timeLimitMinutes
attemptLimit
shuffleQuestions
shuffleOptions
showResultImmediately
showCorrectAnswersAfterSubmission
createdAt
updatedAt

```

18.3. Timer and Attempt Controls

- Countdown timer visible to member.
- Auto-submit when timer ends.
- Warning when 1 minute remains.
- Prevent changing answers after final submission.
- Prevent multiple active attempts if attempt limit is one.
- Store startedAt on server/database, not only browser timer.
- Support resume if browser refreshes by calculating remaining time from startedAt.
- Save submittedAt, timeTakenSeconds, autoSubmitted, and attemptStatus.

18.4. Quiz Attempt Fields

```

attemptId
quizId
memberId
answers
score
percentage
result
startedAt
submittedAt
timeLimitMinutes
timeTakenSeconds
autoSubmitted
attemptStatus

```

19. Dashboards and Reports

19.1. Member Dashboard

- Today's attendance status.
- Submit today's sadhana.
- My current sadhana streak and history.
- My assigned seva, due today, in progress, and completed.
- Submit seva update.
- New/active quizzes.
- Visible resources.
- My group details and announcements.

19.2. Leader Dashboard

- Today's attendance marking and absent members.
- Members absent repeatedly.
- Today's sadhana submissions and pending submissions.
- Members needing help.
- Assigned seva list, pending updates, pending reviews, blocked sevas.
- Total seva hours by group.
- Pending one-to-one calls and follow-ups due today.
- New members assigned for approval.

19.3. Admin Dashboard

- New signups waiting for approval.
- Daily attendance summary.
- Daily sadhana summary.
- Seva assignment summary and total seva hours.
- Pending seva reviews and blocked sevas.
- Group-wise and zone-wise performance.
- Inactive members and members needing help.
- Overdue follow-ups and leader activity status.
- Draft/published resources, quizzes, and meeting action items.

19.4. Required Reports

- Member report, group health report, daily attendance report, daily sadhana report, follow-up report, call report, seva report, quiz report, leader activity report, inactive member report, new member onboarding report, meeting action item report, resource acknowledgement report.
- Admins should export CSV reports. Leaders/members should not export sensitive data.

19.5. Summary Collections for Cost Control

dailyGroupSummary
 dailyZoneSummary
 monthlyMemberSummary
 monthlyGroupSummary
 leaderActivitySummary
 sevaContributionSummary
 quizPerformanceSummary

- Dashboards should read summary documents whenever possible instead of scanning thousands of raw daily records.

20. Notifications and WhatsApp Integration

20.1. Notification Channels

In App
Email
WhatsApp
Push (future)

20.2. Notification Preferences

whatsappRemindersEnabled
emailRemindersEnabled
sessionRemindersEnabled
sadhanaRemindersEnabled
sevaRemindersEnabled
quizRemindersEnabled
preferredReminderTime
quietHoursStart
quietHoursEnd

20.3. WhatsApp Group Link Automation

- After final approval, fetch assigned group WhatsApp link and send an automated message if WhatsApp API/provider is configured.
- Store whatsappInviteSent and whatsappInviteSentAt.
- If automation fails or API is not ready, create a manual send task.
- MVP must include Generate WhatsApp Message, Copy Message, Open WhatsApp, and Mark as Sent.
- WhatsApp group links must be hidden from unapproved users.

20.4. WhatsApp Invite Message Template

Hare Krishna [Name] 🙏

Welcome to Prabhupada World Bhakti Vriksha.

Your registration has been approved, and you have been assigned to [Group Name].

Please join your Bhakti Vriksha WhatsApp group using the link below:

[WhatsApp Group Link]

We are happy to have you with us.

Your servant,
Prabhupada World Team

21. Firestore Collections and Data Model

Use top-level collections for logs and reports to keep querying practical. Use subcollections only where they do not hurt reporting. Store IDs like memberId, groupId, zoneId, and date on daily logs for efficient queries.

21.1. Full Collection List

users
members
groups
zones
sessions
dailyAttendance
dailySadhanaLogs
callLogs
followUps

remarks
 quizzes
 quizQuestions
 quizAttempts
 sevaCategories
 sevaAssignments
 sevaLogs
 meetings
 meetingMinutes
 meetingActionItems
 resources
 resourceCategories
 notifications
 auditLogs
 memberTimeline
 groupTransferHistory
 temporaryAccess
 resourceAcknowledgements
 summaryCollections
 appSettings

21.2. Database Design Principles

- Use Firestore-generated IDs unless a deterministic ID prevents duplicates.
- Recommended deterministic IDs: dailySadhanaLogs/{memberId_YYYY-MM-DD} and dailyAttendance/{sessionId_memberId}.
- Every important collection should store createdAt, createdBy, updatedAt, updatedBy.
- Use soft delete with isDeleted/archived instead of permanent deletion.
- Use denormalized display fields only where needed for dashboard performance.
- Never rely on frontend-only checks for data access.
- Use composite indexes for groupId + date, memberId + date, zoneId + date, status + dueDate, and visibility queries.

21.3. Search and Filters

- Search members by name, phone, WhatsApp number, email, city, group, status, leader, mentor, seriousness level.
- Filter by status, city, zone, group, membership type, current stage, sadhana submitted/not submitted, attendance status, follow-up pending, quiz completed/not completed, seva status.
- Search meeting minutes by meeting type, date, discussion keyword, decision, action owner.
- Search resources by title, type, category, visibility, and latest version.

22. Automations

Automation	Trigger	Action
New signup submitted	Profile submitted after email verification	Set Pending Admin Review, notify admin, show waiting page, create audit log.
Admin assigns member	Admin selects group and leader	Set Assigned to Leader, notify leader, show approval request.
Leader accepts member	Leader clicks accept	Set Approved/Active, grant dashboard access, send WhatsApp group link or manual task.
Attendance not marked	Daily scheduled job	Notify leader and coordinator dashboard.
Consecutive absence	Attendance marked/daily job	Create follow-up at 2 absences, escalate at 3+ absences.
Sadhana not submitted	Daily scheduled job after grace period	Show pending status, notify member/leader as configured.
Needs help in sadhana	Sadhana log submitted with needsHelp	Create follow-up for leader/mentor.
Seva assigned	New seva assignment	Notify member and show on dashboard.
Seva not started/due/overdue	Daily scheduled job	Notify member/leader and update dashboard.
Seva blocked	Seva log blocked or needsHelp	Notify leader and create follow-up.

Seva submitted for review	Seva log status Submitted	Notify leader and show Pending Review.
Quiz published	Quiz Published and isVisible true	Show on assigned member dashboards and notify members.
Quiz timer ends	Server/database calculated expiry	Auto-submit and calculate score.
Meeting action item due	Daily scheduled job	Notify owner and dashboard.
Resource published	Resource status Published	Show to visible audience and optionally notify.
Weekly admin report	Scheduled weekly job	Generate attendance/sadhana/seva/follow-up/group health summary.

23. Security, Audit Logs, Backup and Restore

23.1. Security Requirements

- All users must be authenticated before accessing internal data.
- Use role-based route protection and Firestore Security Rules.
- Enable Firebase App Check.
- Use backend validation for sensitive writes.
- Use environment variables and Cloudflare/Firebase secrets for API tokens.
- Never expose Firebase Admin credentials or WhatsApp API secrets in frontend code.
- Members cannot access dashboards, resources, quizzes, or group data until approval is complete.
- Private remarks and counselling notes must have strict visibility controls.

23.2. Audit Logs

```
auditId
actorUserId
action
entityType
entityId
oldValue
newValue
timestamp
ipAddress
userAgent
reason
```

- Log member created/updated/archived, group changes, role changes, approval actions, private remarks, attendance edits, sadhana edits, seva completion, resource publishing, quiz publishing, meeting minute approval, data export, and bulk imports.
- For sensitive fields, also keep edit history with old value, new value, who changed it, when, and why.

23.3. Backup and Restore

- Create daily backups for critical collections.
- Create manual backup before bulk import or bulk update.
- Provide restore procedure for accidental deletion or wrong import.
- Use soft delete instead of hard delete.
- Critical collections: members, groups, attendance, dailySadhanaLogs, callLogs, followUps, sevaAssignments, sevaLogs, meetingMinutes, resources, quizAttempts.

24. Operational Safeguards and Loophole Fixes

Safeguard	Why it matters
Duplicate detection and merge	Prevents the app from becoming another messy master sheet with repeated members.
Member timeline	Gives leaders a complete journey history in one place.
Group transfer history	Preserves why and when members shifted groups.
Temporary access	Avoids password sharing when substitute leaders help.

Notification preferences and quiet hours	Prevents reminder fatigue and irritation.
WhatsApp manual fallback	Prevents launch dependency on WhatsApp API approval.
Server-side quiz timer	Prevents timer misuse through browser refresh or manipulation.
Grace periods	Avoids false missing alerts for attendance/sadhana.
Correction workflow	Allows data correction without casual overwrites.
Resource acknowledgement/versioning	Ensures leaders are using the latest scripts/guidelines.
Meeting decision log	Answers who decided what and when.
Summary collections	Controls Firebase read costs for daily dashboards.

24.1. Duplicate Detection Rules

- Check phone number, WhatsApp number, email, full name + phone similarity, and full name + city.
- Admin options: approve as new, merge with existing, mark as duplicate, reject duplicate.
- Merge actions must be audit logged.

24.2. Data Correction Workflow

- Allow Request Correction, Approve Correction, Correction Reason, and Correction History.
- Older attendance/sadhana corrections should require elevated permission.
- Bulk corrections should require admin confirmation and backup.

25. MVP Scope and Future Roadmap

25.1. MVP Build Order

1. Firebase setup, Firestore, Auth, email verification, environment secrets.
2. PWA shell, login, signup, and Waiting for Approval screen.
3. Roles, route protection, Firestore Security Rules, App Check.
4. Admin approval dashboard and leader acceptance workflow.
5. Member profile and group/zone management.
6. Daily attendance module.
7. Daily sadhana module.
8. Seva assignment and seva logs.
9. One-to-one call logs and follow-ups.
10. Resources module with visibility.
11. Quiz publishing, timer, attempts, and results.
12. Meetings and minutes with action items.
13. Dashboards, reports, CSV import/export.
14. Audit logs, backups, summary collections, final testing.
15. Cloudflare deployment.

25.2. Future Scope

- WhatsApp API automation beyond manual fallback.
- Course progress integration with Jigyasa and other PW courses.
- Calendar integration for sessions, meetings, seva, and follow-ups.
- Spiritual progress milestones and badges.
- Certificates/appreciation for quiz, sadhana consistency, and seva.
- Multi-language support.
- AI summaries for member journey, calls, meetings, and leader reports.
- Native mobile app if needed later.

26. Development Instructions for Antigravity

26.1. General Build Instructions

- Build as a production-quality React or Next.js TypeScript app.
- Use Firebase Auth, Firestore, Firebase Security Rules, Firebase Cloud Functions, and PWA support.

- Use Cloudflare Pages-compatible deployment.
- Use reusable components, clean folder structure, loading states, empty states, error handling, and confirmation modals.
- Use environment variables and never hardcode secrets.
- Include seed/test data for development.
- Implement role-based route protection and Firestore rules before broad data features.

26.2. Suggested Folder Structure

```

/src
  /components
  /pages or /app
  /routes
  /features
    /auth
    /signupApproval
    /members
    /groups
    /attendance
    /sadhana
    /seva
    /calls
    /followups
    /meetings
    /resources
    /quizzes
    /reports
    /admin
  /services
    firebase.ts
    authService.ts
    memberService.ts
    groupService.ts
    attendanceService.ts
    sadhanaService.ts
    sevaService.ts
    callService.ts
    followupService.ts
    meetingService.ts
    resourceService.ts
    quizService.ts
  /hooks
  /utils
  /types
  /styles
  /firebase
    firestore.rules
    firestore.indexes.json
  /functions
  /cloudflare
    workers/

```

26.3. Reusable Components

- Layout, sidebar, top bar, dashboard cards, data table, search bar, filters, forms, status badges, role badges, loading spinner, empty state, confirmation modal, access denied page.
- Specific forms: member form, approval form, group form, attendance form, sadhana form, seva assignment form, seva log form, call log form, follow-up form, meeting minutes form, resource upload form, quiz builder, quiz attempt screen with timer.

27. Acceptance Criteria

16. User can install/open the web app.
17. User can sign up, verify email, fill personal and sadhana details, and submit profile.
18. After submission, user sees Waiting for Approval and cannot access internal features.
19. Admin can view new signup, detect duplicates, and assign member to group leader.
20. Group leader can accept/reject/on-hold assigned member.
21. After approval, member becomes active and receives WhatsApp group link automatically or manual send task is created.
22. Admin can create users, groups, zones, sessions, resources, quizzes, meetings, and reports.
23. Leader can mark daily attendance for assigned group and member can view own attendance.
24. Member can submit daily sadhana; duplicate same-day logs are prevented.
25. Leader can view daily sadhana of assigned group and follow-up is created when help is requested.
26. Leader can assign seva; member can update seva status, time spent, progress, blockers, and remarks.
27. Leader can review seva, request changes, and mark completed.
28. Call logs and follow-ups can be created and tracked with owners and due dates.
29. Meeting minutes and meeting action items can be stored, searched, and assigned.
30. Resources can be uploaded with visibility control, versioning, and acknowledgement.
31. Quiz tab is visible but unpublished quizzes are hidden.
32. Leader/admin can publish quiz; published quiz appears immediately to assigned members.
33. Quiz supports countdown timer and server/database-based auto-submit.
34. Dashboards show daily attendance, daily sadhana, seva, pending approvals, follow-ups, resources, quizzes, and meeting action items.
35. Role-based access prevents unauthorized data access at both frontend and Firestore rules levels.
36. Firebase App Check, audit logs, backups, soft deletion, and environment secrets are implemented.
37. CSV import/export works for admin with duplicate warning and import logs.
38. App works properly on mobile and desktop.

28. Appendices: Status Values and Master Prompt Summary

28.1. Master Status Values

Area	Values
Approval	Draft, Submitted, Email Verified, Pending Admin Review, Assigned to Leader, Pending Leader Approval, Leader Accepted, Approved, Rejected, On Hold.
Member	New, Active, Needs Follow-up, Temporarily Paused, Inactive, No Response, Not Interested, Shifted, Duplicate, Archived, Reactivated.
Attendance	Present, Absent, Late, Excused, Unknown.
Sadhana Visibility	Member and Leader, Leader Only, Admin Only.
Seva	Assigned, Not Started, Started, In Progress, Submitted, Needs Review, Changes Requested, Completed, Blocked, Paused, Cancelled.
Follow-up	Pending, Completed, Overdue, Cancelled.
Meeting Minutes	Draft, Submitted for Approval, Approved, Published, Archived.
Resource	Draft, Published, Archived, Expired.
Quiz	Draft, Published, Unpublished, Closed, Archived.
Quiz Attempt	In Progress, Submitted, Auto Submitted, Expired, Cancelled.

28.2. Condensed Master Prompt for Antigravity

Build a production-quality installable Progressive Web App named PW Bhakti Vriksha using React/Next.js TypeScript, Cloudflare Pages, Firebase Auth, Firestore, Firebase Cloud Functions, Firebase Security Rules, and Firebase App Check. The app must manage self-signup with email verification, Waiting for Approval state, admin assignment, leader acceptance, WhatsApp group link sending/manual fallback, member profiles, zones/groups, daily attendance, daily sadhana, seva assignment and seva logs, one-to-one calls, follow-ups, meeting minutes/action items, resources with visibility/versioning/acknowledgement, quizzes with publish control and countdown timer,

dashboards, reports, audit logs, duplicate detection/merge, group transfer history, temporary access, notification preferences, backup/restore, correction workflow, and summary collections for cost control. Enforce role-based access at UI, backend, and Firestore Security Rules levels. Do not expose secrets in frontend. Members must not access internal data until approved.

Final development caution

Do not ask Antigravity to code the entire app in one prompt. First ask it to produce database schema, route map, security rules plan, and implementation milestones. Then build module by module.